Open questions

Resolved

- **Subnet numbering.** Spec said 12 theatres on "192.168.2-12.x", which is only 11 subnets. Confirmed mapping: mothership on `.1.x`, Theatre 1 → `.2.x` … Theatre 12 → `.13.x` (contiguous after mothership). Used throughout this repo.
- **Mothership router: Ubiquiti Cloud Gateway.** Cloud Gateway (UniFi-OS-based, same family as the Dream Machine) generally can't run Tailscale natively without an unofficial container hack. Decision: the mothership subnet-router role runs as a container on the consolidated services server instead of the Cloud Gateway itself — see `docs/topology.md`.
- **Mothership service consolidation: single physical server running Unraid.** Nextcloud, Restreamer, BirdDog Central, and the NDI Discovery Server all move onto one box instead of being separate machines — the router and the 4 ready-room PCs stay physical. BirdDog Central is Windows-only, so it runs as the design's one Windows VM — routing/control, not encode, no GPU needed. (An earlier revision also carried a mothership VMix instance VM with a passthrough GPU; it was removed — see #10 below.) Everything else (Nextcloud, Restreamer, NDI Discovery Server, Tailscale subnet router) runs as Docker containers. Chose **Unraid over TrueNAS SCALE** for its polished one-click container experience (the original GPU-passthrough-maturity tiebreaker no longer applies with the VMix VM gone), accepting the $129 one-time Lifetime license cost that TrueNAS (free) doesn't have. See `docs/topology.md` for the full breakdown.
- **Theatre router: GL-iNet A-1300 (Slate Plus).** Published client-mode WireGuard throughput ≈ 170 Mbps (GL.iNet A-1300 datasheet). Each theatre's router only ever carries *its own* theatre's traffic — one incoming SRT feed to that theatre's BirdDog Play plus up to 4 laptops' rclone sync — not the aggregate across all 12 theatres (that 12x aggregate converges at the mothership's WAN link and Restreamer, not at any single theatre router). So the real per-router budget is roughly 1× SRT bitrate (~8–50 Mbps depending on resolution/codec) + bursty rclone traffic from up to 4 laptops — comfortably inside the A-1300's 170 Mbps headroom. *(Caveat: that traffic list predates ATEM ISO ingest and the Overseer/Flock monitoring streams, all of which now ride the same router. The current per-router model — 97% of the ceiling at worst-case normal load, 128% during an NDI fallback with ingest running — is in `docs/bandwidth-analysis.md`'s "check the A-1300's own ceiling" section; the router choice still stands, but "comfortably" no longer describes the margin.)*
- **VMix node routers: also GL-iNet A-1300.** Same model as the 12 theatre routers — 14 A-1300s total across the network. Both LAN ports bridge together (no dedicated-port split needed here, unlike the theatre routers' BirdDog Play case). Configs generated: `config/gl-inet/vmix-node-1.uci` and `vmix-node-2.uci`.
- **NDI backup path discovery.** Confirmed Tailscale can't help directly — it's point-to-point WireGuard and doesn't forward multicast/mDNS (a still-open Tailscale feature request, not something MagicDNS

works around). Plan: run an **NDI Discovery Server** on the mothership (unicast TCP, default port 5959 — built into NDI 5+ specifically for WAN/VPN cases like this), and point BirdDog Central plus all 12 BirdDog Play units at it. See `docs/streaming-flow.md` for the full plan.

- **BirdDog Play Discovery Server support — confirmed.** BirdUI (PLAY's web admin) has a Network panel with an NDI Discovery Server toggle: switch it on, enter a comma-delimited list of server IP(s), apply. BirdDog's own docs describe this as the mechanism for finding sources "on different subnets." Sources: BirdDog PLAY User Guide, BirdUI User Guide. Default admin password (`birddog`) should be changed on all 12 units before the event.

- **Consolidated server: hardware already on hand.** Physical box already exists — no procurement needed. Known so far: 2× gigabit NICs, a "decent" discrete GPU. Full spec (CPU/motherboard, RAM, exact GPU model) to be confirmed later — a calculated **target** minimum spec to check it against (storage pool layout/sizing, RAM, CPU, GPU tier, NIC validation, all worked from this box's actual workload rather than guessed) is now in `docs/server-specification.md`.

- **Full device inventory + configs generated.** Every device on the network now has a concrete IP — see `docs/ip-address-map.md` (133 devices total) and the redrawn `diagrams/topology.svg`. Configs generated to match: `config/gl-inet/` (UCI network config for all 14 routers — 12 theatres + 2 VMix nodes), `config/tailscale-up-all-devices.sh` (all 15 tailnet nodes instantiated), and `config/unifi/` (Cloud Gateway VLANs/DHCP/firewall reference).

  Two things introduced *during config generation* that weren't decided before — flagged here since they're new, not previously confirmed:

  - **Uplink VLAN transit addressing** (`10.10.1-4.0/24`, VLAN tags 101–104) for the 4 physical VLAN groups' GL-iNet WAN ports. Doesn't affect anything inside the theatres (still NAT'd behind each A-1300) or Tailscale (rides over whatever WAN IP is handed out) — purely a Cloud Gateway-side detail. Change freely.
  - **Docker networking mode per mothership service**: Nextcloud/Restreamer/NDI Discovery Server/DERP server assigned dedicated IPs via macvlan; Tailscale subnet router uses host networking (required for route advertisement, shares the Unraid host's `.2`). Both are standard Unraid patterns, not unusual choices, but worth confirming once you're actually configuring Docker on the box.

- **2 gigabit NICs: bonded (802.3ad/LACP), not split.** SRT bandwidth isn't constant, and this box handles many simultaneous flows (12 theatres' SRT fan-out to distinct destinations, plus rclone/Nextcloud/BirdDog Central/NDI Discovery Server/DERP traffic) — LACP's hash-based distribution spreads those across both links for real aggregate headroom, rather than a static split that leaves one NIC idle whenever traffic doesn't match the assumed pattern. See `docs/topology.md` for the full reasoning, the Unraid setup steps, and the LACP rate/hash-policy gotcha with Ubiquiti gear.

- **Self-hosted DERP server added.** Runs as a Docker container on the Unraid box (`192.168.1.16`), registered with the tailnet as DERP region 900, `RegionCode: "example"`, hostname `derp.example.net`, port **443** (matches Tailscale's own DERP fleet, most firewall-friendly choice) — see `config/tailscale-acl.json`. Kept `OmitDefaultRegions: false` so Tailscale's public DERP regions remain available as a fallback if this one goes down. See `docs/tailscale.md` for the full setup (container flags, port forward, cert handling).

- **Cloud Gateway assumed to support LAG.** Proceeding on that assumption rather than confirming the exact model first. Fallback already agreed if it turns out not to: drop an intermediate LACP-capable switch in between and bond the Unraid server's NICs through that instead of directly into the Cloud Gateway — the bond itself doesn't care which device terminates it, only that *something* in the path speaks LACP.

- **ATEM ISO ingest architecture decided, two documented approaches.** Each ATEM's built-in FTP server (confirmed live-accessible during recording) is pulled continuously by a new Unraid container (`192.168.1.17`) over Tailscale subnet routing. Plain rsync can't connect to the ATEM at all (no SSH/rsync daemon), and a naive whole-file re-sync is ruled out by the bandwidth math for any real session length — so genuinely incremental transfer is required, achieved either via **FTP's `REST` /resume command directly** (`pull-iso.py`, the default — fewer moving parts) or via **`rclone mount` + `rsync --append`** (`rclone-mount-rsync/` — off-the-shelf tools, at the cost of 12 FUSE mounts to supervise). Both write directly into Nextcloud's External Storage (Local) mount, so there's no separate upload step and no need to slice/chunk the video file itself either way. Full comparison in `docs/atem-iso-ingest.md`.

- **VMix record ingest added, same mechanism as the ATEM ingest.** All 4 VMix PCs' recordings pulled by a new Unraid container (`192.168.1.18`) via `rclone mount` + `rsync --append`, targeting each PC's Windows SMB share instead of an FTP server. SMB is a native network filesystem protocol, so this is arguably a better fit for a mount-based approach than the ATEM's FTP-only case was, not a stretch of the same pattern. Each VMix node has its own dedicated A-1300 uplink (not shared with the theatre it sits near), so this traffic never competes with that theatre's ATEM ingest or SRT feed. Full design in `docs/vmix-record-ingest.md`, tooling in `config/vmix-record-ingest/`.

- **Self-hosted UniFi Controller added to the consolidated server.** A UniFi Network Application container (`192.168.1.19`) joins the rest of the Docker stack, same self-hosted-over-cloud.ui.com rationale already used for DERP. The Cloud Gateway has its own built-in controller and doesn't require this — adopting it in is optional, purely a local/independent management console, not a change to any traffic path in `config/unifi/network-config.yaml`. See `config/unifi/README.md`.

- **Self-hosted GLKVM-Cloud added for centralized administration of the 14 GL-iNet routers.** No physical KVM hardware involved — this uses GLKVM-Cloud's separately documented HTTP/HTTPS web-proxy and device-management support for embedded OpenWrt devices (which the A-1300s are), giving one browser-based SSH terminal + web-admin proxy for all 14 routers instead of 14 separate sessions. Same self-hosted-over-vendor-cloud rationale as DERP and the UniFi Controller, avoiding GL.iNet's own `glkvm.com`. Two containers (`rttys` on `192.168.1.20`, `coturn` on `192.168.1.22`); router registration rides the existing tailnet (routers already accept routes to `192.168.1.0/24`), no WAN exposure needed for that part. Surfaced a real conflict during design: GLKVM-Cloud's documented ports (`443/tcp`, `3478/tcp+udp`) collide with the DERP server's existing WAN forwards on those same numbers — resolved by forwarding different WAN-side port numbers (`8443`, `3479`) to GLKVM-Cloud's unchanged internal ports (for the admin's own browser access), rather than moving DERP off 443 and losing its firewall-friendliness rationale. Not yet confirmed against a real deployment: whether `GLKVM_ACCESS_IP` (the env var that tells the app what address to hand back to clients) accepts this WAN-side remap cleanly, or whether `coturn` needs its own separate external-address setting — also unclear whether `coturn`/TURN is needed at all for the SSH-terminal/web-proxy features actually in use here, versus only for GLKVM-Cloud's KVM-specific

remote-desktop feature, which doesn't apply to routers. Full design in `docs/glkvm-cloud.md`, stack in `config/glkvm-cloud/`.

- **Bandwidth modeled against venue-supplied VLANs — 2 VLANs (not 4) recommended, but reconsider given how thin that margin has gotten.** The 4 uplink VLANs turned out to be venue-supplied, 1 Gbps each, with expensive ports — changes the goal from "isolate cleanly" to "minimize VLAN count at adequate performance." Real numbers (last updated after adding the Flock BirdDog Play preview stream, ~10.4 Mbps/theatre, on top of ATEM Overseer's — see below): per-theatre upstream is dominated by ATEM ISO ingest (~62 Mbps realistic of ~88 Mbps total) — at the current 3-theatres-per-VLAN split, that's ~27% utilization, with room to consolidate to 2 VLANs (6 theatres each, ~53% realistic / 78% worst-case) before it stops being comfortable; **1 VLAN for all 12 theatres now exceeds capacity even at realistic load (106%)**, not just worst case — it's only viable at all if ATEM ingest is deferred to end-of-session pulls instead of real-time. Full model in `docs/bandwidth-analysis.md`.

- **ATEM Overseer + ATEM Fleet Admin added to the mothership.** Two of the user's own separate projects, deployed as Docker containers (`192.168.1.21` and `192.168.1.23`) alongside everything else — Overseer for fleet-wide monitoring/tally (every theatre's ATEM sends it a dedicated 10 Mbps monitoring stream, folded into the bandwidth model above as a flat per-theatre addition that doesn't scale with ATEM channel count), Fleet Admin for bulk provisioning (reaching all 12 theatres' ATEMs the same way ATEM ISO Ingest does, over the existing Tailscale subnet routing). Neither is built from source in this repo — `config/docker-compose.yml` uses placeholder image references pending confirmation of each project's actual published container image.

- **Flock added to the mothership.** A third of the user's own projects, deployed alongside the other two (`192.168.1.24`) — the BirdDog Play fleet manager already referenced elsewhere in this design (LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits — see `docs/birddog-play-rationale.md`). Each theatre's BirdDog Play sends Flock a 10 Mbps SRT preview stream (confirmed by the user, not assumed), folded into the bandwidth model the same way as the Overseer stream — another flat per-theatre addition. Same not-built-from-source caveat as Overseer/Fleet Admin above.

- **Live editing subsystem added — new `192.168.22.0/24` subnet, own 10GbE LAN, not on the Tailscale mesh.** 2× MacBook Pro editing workstations plus dedicated fast storage at the mothership, editing event content as it arrives rather than after the event. Deliberately separate from the theatre-facing network — different traffic pattern (sustained multi-gigabit editing I/O vs. the bursty low-bitrate streams everything else here is sized for) and a genuinely different failure domain. Key decisions made: dedicated NAS (not Nextcloud's own storage), dual-write from the same rclone/rsync pull already built for ingest rather than a chained re-sync; a dedicated Mac mini running both the Resolve Project Server and Remote Render (needed because resolve-configurator builds one shared show project both editors work against, not independent copies — confirmed **not** possible on a Blackmagic Cloud Store appliance itself, storage-only, no general compute); Blackmagic Cloud's internet sync service not needed for a single-venue setup. Full design, including the ATEM-pull-to-finished-edit workflow and the DIT physical-media ingest tool comparison (ShotPut Pro/OffShoot vs. Silverstack vs. FoolCat/o/PARASHOOT as companion tools), in `docs/live-editing.md`.

Still open — verify before building

1. **Check the existing server's real spec against the calculated target in `docs/server-specification.md`.** In particular: does it have validated (not merely tolerated) ECC support, enough PCIe lanes for the three NVMe pools, and enough RAM/cores (target: 12 cores/24 threads, 64-96 GB RAM) to run the BirdDog Central Windows VM plus 12 Docker containers concurrently without contention during a live event. (GPU passthrough/IOMMU checks dropped from this list — nothing needs them since the VMix VM was removed, see #10.)

2. **Actually register/point `derp.example.net`** (or substitute whatever domain/DDNS host you end up controlling) at the mothership's public IP, and confirm the WAN port forward (443/tcp, 3478/udp) is in place before relying on it — the name and port are picked, but nothing resolves until the DNS record and port forward both exist.

3. **Verify the DERP hairpin path doesn't get caught by the uplink-VLAN firewall block.** `config/unifi/network-config.yaml` blocks the 4 uplink VLANs from reaching Mothership-LAN directly (defense-in-depth for cross-theatre isolation) — that should be a different path from the DERP hairpin (which arrives via the WAN zone after NAT, not directly from an Uplink-VLAN zone), but exact zone/hairpin behavior is Cloud-Gateway-firmware-specific. Confirm with `tailscale netcheck` on a theatre router once built, before assuming the self-hosted DERP server is actually reachable.

4. **Confirm the real ATEM ISO bitrate/active-channel count, and empirically verify partial-file playability.** The bandwidth plan above uses 10 Mbps/stream and an assumed 4-6 active channels — worth checking against actual ATEM recording settings. Whether a file copied mid-recording is genuinely valid (rather than just "very likely, given Blackmagic's marketed edit-while-recording capability") should be tested against a real unit before relying on it operationally. See `docs/atem-iso-ingest.md` for the full open-items list, including confirming the ATEM's actual FTP credentials/file layout and tuning the pull/scan intervals and Nextcloud version-retention settings once this is running for real.

5. **Confirm what VMix is actually recording, and set up real SMB shares/credentials.** Neither the recording mode (program mix vs. per-input ISO) nor bitrate/codec, nor the actual share name/folder path/account on the 4 VMix PCs, is assumed here — see `docs/vmix-record-ingest.md` for the full open-items list.

6. **Confirm with the venue: final VLAN count, the full-duplex-1Gbps assumption, and whether presenter internet shares the production VLANs or is separate infrastructure.** `docs/bandwidth-analysis.md` recommends 2 VLANs over the current 4, but that's a real cost/procurement decision, not something this repo can finalize alone. If real-time ATEM ingest turns out incompatible with the venue's final VLAN allocation, deferring it to end-of-session pulls is the fallback — same doc.

7. **Confirm `GLKVM_ACCESS_IP`'s exact accepted format, and whether `coturn` needs its own separate external-address setting**, before trusting the WAN-remapped port forwards (`8443`, `3479`) to actually work end-to-end for remote GLKVM-Cloud admin access. See `docs/glkvm-cloud.md` for the fallback (second public IP or an SNI-routing reverse proxy) if the plain env var doesn't cover it. Also confirm whether `coturn` is needed at all for the SSH-terminal/web-proxy features actually in use here — it may only matter for GLKVM-Cloud's KVM-specific remote-desktop feature, unused since there's no KVM hardware in this design.

8. **New assumptions introduced by `config/docker-compose.yml`**, none confirmed against a real deployment: the NDI Discovery Server image (`pnxr/ndi-discovery-minimal`, a third-party community build, not NDI/NewTek-published — confirm it's still maintained), the MongoDB version the UniFi Controller needs (drifts with the controller image's own version, not fixed here), and MariaDB + Redis as Nextcloud's DB/cache backend (chosen over SQLite — this box has a lot of concurrent writers across 12 theatres' rclone syncs plus both ingest pipelines, which official Nextcloud guidance says SQLite handles poorly; not benchmarked here either way).

9. **Confirm actual event duration (day count × active-recording hours/day)** before treating the 8 TB recording pool in `docs/server-specification.md` as settled — that document calculates it covers roughly 15.5-23.5 hours of continuous worst-to-realistic-case load (~1.5-3 typical event days), but this repo doesn't establish the event's real length anywhere. If it runs longer without a periodic archive-off step, either the pool needs to grow or an offload step needs adding to the ingest design.

10. **Resolved — by removal.** This slot used to ask what the mothership's own VMix instance VM actually did (its role was never established anywhere in this repo, unlike the well-documented VMix *node* PCs). The answer arrived as a design change: **the mothership VMix VM was removed entirely.** The theatre program feeds originate from the VMix node PCs via Restreamer (`docs/streaming-flow.md`), which never depended on it. Knock-ons all applied: no GPU/passthrough/IOMMU requirement remains anywhere in the spec, RAM/CPU budgets dropped (see `docs/server-specification.md`), BirdDog Central is now the design's only VM, and the device count is 133. (Number kept so cross-references to #11-#15 stay stable.)

11. **Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's actual published container images** (or that they need to be built from source instead) before deploying `config/docker-compose.yml` — all three use placeholder `ghcr.io/...` references. Also confirm the actual mechanism behind both monitoring streams `docs/bandwidth-analysis.md` takes as given inputs, not verified against real hardware: Overseer's 10 Mbps ATEM stream (does the ATEM Mini Extreme ISO genuinely support two independent simultaneous stream outputs — its own hardware streaming engine feeding both Restreamer and Overseer at once?), and Flock's 10 Mbps BirdDog Play preview stream (same question for BirdDog Play — decoding its primary program feed while simultaneously *originating* a second SRT stream back to Flock is a different capability than the receive-only role it plays everywhere else in this design).

12. **Decide how much of the event's footage the live-editing subsystem actually needs access to** — the full 8TB recording pool, or a curated subset — before sizing the edit-suite NAS in `docs/live-editing.md` Decision 1.

13. **Confirm combining Project Server and Remote Render on one Mac mini holds up under real load** — `docs/live-editing.md` Decision 2 treats this as architecturally sound but thinly documented by both Blackmagic and practitioners; worth a real pre-event test, not just a design-time assumption.

14. **Confirm whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the DIT physical-media ingest workflow in `docs/live-editing.md` has real camera cards to process beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data feature set becomes worth its cost over ShotPut Pro/OffShoot.

15. **Implement the second write destination in the actual ingest scripts.** `docs/live-editing.md` documents `pull-iso.py` and `mount-and-sync.sh` writing to the edit-suite NAS alongside Nextcloud — that's a real code change neither script has yet (`config/atem-iso-ingest/pull-iso.py` , `config/vmix-record-ingest/mount-and-sync.sh` currently write one destination each). Not done as part of documenting the design.

16. **Verify the settled NDI-redistribution feed into BirdDog Central.** The mechanism is decided (`docs/streaming-flow.md`): the source VMix node PC's native NDI output → NDI Discovery Server registration → Central receives over that node's uplink (fallback-only ~130 Mbps against the ~170 Mbps ceiling, with that node's record ingest paused for the duration) → Central re-sends to the theatre PLAYs. The alternatives were researched and ruled out: Restreamer cannot emit NDI (upstream FFmpeg removed NDI in 2019 over a NewTek GPL violation; datarhei confirmed no NDI in their build and declined on licensing grounds — discussion #391), and Central cannot ingest SRT (NDI-only per its own user guide). What remains is verification, not design: confirm VMix's NDI output registers with the discovery server and Central picks it up across the tailnet; confirm Central's re-transmit actually re-originates the stream (rather than pointing receivers at the original source, which would bypass the mothership and break both the ACL model and the bandwidth math); and measure the node uplink during a rehearsed fallback against the modelled ~130 Mbps.